大语言模型综述

第 1 章

大语言模型定义 · 缩放定律 · 涌现能力 · 分词与词嵌入

著者
邓杭　杨耀东

教材
《大语言模型：训练、精调与对齐》

出版
清华大学出版社



本章目录

1.1 **大语言模型介绍** — 概念、历史与模型脉络 — 第 3 页

1.2 **大语言模型特点** — 缩放、涌现与苦涩的教训 — 第 12 页

1.3 **大语言模型研究现状** — 从语言建模到架构、技术路线与训练数据 — 第 21 页

1.4 **自然语言数据处理** — 分词、词嵌入、Word2Vec 与词嵌入层 — 第 45 页

1.5 **小结** — 本章要点回顾与后续章节的路线 — 第 65 页

阅读建议 先建立“规模与能力”的直觉（1.2），再理解语言建模与架构（1.3），最后动手体验分词与词嵌入（1.4）。

1.1

大语言模型介绍

先界定大语言模型是什么，再沿 Word2Vec、Seq2Seq 与注意力机制的脉络，看 Transformer 如何成为主流架构。

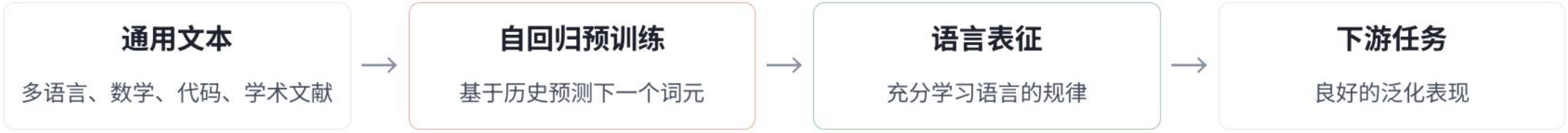
本节内容

1.1.1 大语言模型定义

1.1.2 大语言模型历史

以通用预训练学习语言表征

大语言模型（LLM）是一种通用的语言模型：用大规模参数的神经网络，对通用文本做自回归预训练，学到的语言表征可以泛化到下游任务。



- 语言模型的学习，从“对具体任务的学习”转变为“对通用语言的预训练学习”。
- LLM 主要使用纯解码器（Decoder-Only）架构的 Transformer 模型，通过下一个词预测任务进行自监督学习，无需额外标注。这种方式称为因果语言建模（Causal Language Modeling, CLM）。
- 例如 GPT-2 采用纯解码器 Transformer 与因果语言建模，在通用语料上训练，并泛化到多个语言处理任务，当时表现出色。

“下一个词预测”示例：训练文本“人工智能助手”，输入为“人工智”，标签为“能”。

文献 Radford et al., 2019；Brown et al., *Language Models are Few-Shot Learners*, NeurIPS 2020。

应用场景与规模带来的挑战

LLM 的显著特征是参数数量大。除传统的自然语言任务外，它的应用已扩展到代码、推理、对话、智能体与多模态场景。

基础任务

语言与推理

文本分类、翻译与摘要总结；代码生成、数学解题与逻辑推理。

对话与工具

交互与调用

对话机器人与智能客服；工具使用（Tool-Use），如检索增强生成（RAG）与输出调用 API 的代码。

智能体

智能大脑

接收外部信息，规划与决策智能体行为，与外界交互，完成指定目标。

多模态

视觉与语言

视觉-语言模型（VLM）将视觉信息转化为语义信息；以 LLM 为中心的模型可连通文本、图像、视频与语音。

规模带来的两重挑战

随着参数量与计算规模增加，训练与推理都成为工程难题；这也是后续第 9、10 章的主题。

训练

分布式并行与训练效率

在 GPU 集群上进行高效、稳定的长时间训练，核心是为不同计算任务设计高效的分布式并行策略，以优化集群的整体训练效率（见第 9 章）。

推理

低成本服务

在生产环境中构建高效且低成本的大语言模型服务，实现更快的响应速度和更高的吞吐量（见第 10 章）。

早期方法：Word2Vec 与 Seq2Seq

早期较成功的基于学习的自然语言处理算法是 Word2Vec 与 Seq2Seq。二者分别表征离散的词与离散的词序列，但 RNN 类网络处理长文本时会遗忘信息。

词 → 向量

Word2Vec：表征离散的词

将词映射到嵌入向量空间；学习窗口上下文中的单词关系，根据一个词预测周围可能出现的词，或反过来。

语义相近的词在向量空间中相互接近，从而学到词的表征。

词序列 → 序列

Seq2Seq：表征离散的词序列

编码器把输入序列编码为隐状态（hidden state）向量，解码器再把隐状态向量解码为输出序列。

用于文本摘要、文本翻译与对话生成；常用于特定任务训练，常见模型有 RNN、LSTM 与 GRU。

> **痛点** RNN 类网络在处理较长文本序列时不可避免地出现遗忘问题，导致模型性能下降。

文献 Mikolov et al., 2013（Word2Vec）；Sutskever et al., NeurIPS 2014（Seq2Seq）。

注意力机制与 Transformer

最早的注意力机制旨在解决长序列建模中的信息遗忘问题；Transformer 以多头自注意力，把注意力推向当前语言建模的主流。

RNN 类网络
长序列中易遗忘

→

注意力机制
解码时对输入做软对齐，对隐向量加权

→

相关研究
进一步验证注意力的有效性

→

Transformer（2017）
多头自注意力，提升 Seq2Seq 任务

- 注意力机制对编码阶段的输入序列进行软对齐，对每个时刻的隐向量进行加权处理。
- 谷歌于 2017 年提出的 Transformer 架构具有里程碑意义，它通过多头自注意力显著提升了 Seq2Seq 类任务的性能（输入与输出均为序列，如问答、翻译、摘要），并在当时的文本翻译任务中取得优异结果。

文献 Bahdanau et al., ICLR 2015；Wu et al., 2016；Gehring et al., 2017；Vaswani et al., *Attention Is All You Need*, NeurIPS 2017。

Transformer 架构的三个分支

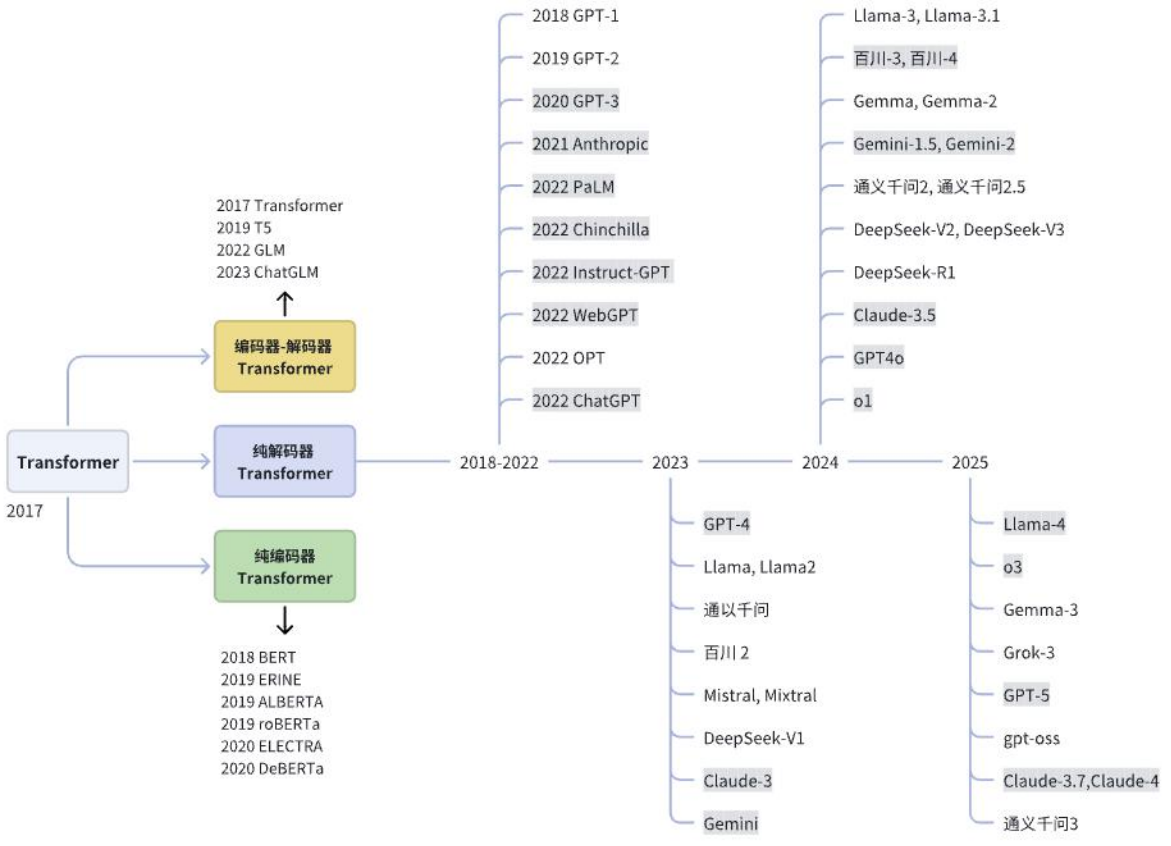


图 1.1　Transformer 架构分枝下的部分模型

编码器-解码器

原始 Transformer 与 T5

编码器-解码器架构常见的有原始 Transformer 模型和 T5 模型。

纯编码器

BERT

纯编码架构常见的有 BERT 模型。

纯解码器

GPT 系列

纯解码架构有 GPT 模型；1.1.1 所述的大语言模型主要采用这一架构。

图中同时标注了各模型的发布年份。三种架构的特点在 1.3.2 节展开讨论。

图源 《大语言模型：训练、精调与对齐》图 1.1

表 1.1　关键语言模型信息（1/2）

模型名称	发布时间	参数规模	模型架构	开源类型
Transformer	2017年06月	65M	编码器-解码器	开源
BERT	2018年10月	110M	编码器	开源
GPT-1	2018年06月	117M	解码器	开源
GPT-2	2019年02月	1.5B	解码器	开源
T5	2019年10月	11B	编码器-解码器	开源
ALBERTA	2019年09月	223M	编码器	开源
RoBERTa	2019年07月	125M	编码器	开源
ELECTRA	2020年03月	102M	编码器	开源
DeBERTa	2020年06月	304M	编码器	开源
GPT-3	2020年05月	175B	解码器	闭源
Gopher	2021年12月	280B	解码器	开源
Anthropic	2021年12月	52B	解码器	开源
LaMDA	2022年01月	137B	解码器	开源

模型名称	发布时间	参数规模	模型架构	开源类型
WebGPT	2021年12月	175B	解码器	闭源
PaLM	2022年04月	540B	解码器	闭源
Chinchilla	2022年03月	70B	解码器	闭源
Instruct-GPT	2022年03月	175B	解码器	闭源
OPT	2022年05月	175B	解码器	开源
GLM	2022年10月	130B	解码器	开源
ChatGPT	2022年10月	-	解码器	闭源
GPT-4	2023年03月	-	解码器	闭源
Llama	2023年02月	65B	解码器	开源
Llama-2	2023年07月	70B	解码器	开源
通义千问	2023年09月	14B	解码器	开源
baichuan2	2023年09月	13B	解码器	开源

注 参数规模单位为百万（M）与十亿（B）；“开源”指公开了模型参数，闭源模型通常通过 API 调用。

表 1.1　关键语言模型信息（2/2）

模型名称	发布时间	参数规模	模型架构	开源类型
Mistral	2023年09月	7B	解码器	开源
Mixtral	2023年12月	8×7B	解码器	开源
DeepSeek-V1	2024年01月	67B	解码器	开源
Claude3	2024年03月	-	解码器	闭源
Gemini	2023年03月	-	解码器	闭源
Llama-3	2024年04月	405B	解码器	开源
baichuan-3	2024年01月	-	解码器	闭源
baichuan-4	2024年05月	-	解码器	闭源
Gemma	2024年02月	7B	解码器	开源
Gemma-2	2024年06月	27B	解码器	开源
Gemini-1.5	2024年9月	-	解码器	闭源
Gemini-2	2023年12月	-	解码器	闭源
通义千问-2	2024年06月	72B	解码器	开源
通义千问-2.5	2024年09月	72B	解码器	开源
DeepSeek-V2	2024年05月	236B	解码器	开源
DeepSeek-V3	2024年12月	671B	解码器	开源
DeepSeek-R1	2025年01月	671B	解码器	开源
Claude-3.5	2024年06月	-	解码器	闭源
GPT4o	2024年05月	-	解码器	闭源
o1	2024年09月	-	解码器	闭源
Claude-3.7	2025年02月	-	解码器	闭源
o3-mini	2025年02月	-	解码器	闭源
QwQ	2025年03月	32B	解码器	开源
Gemma-3	2025年03月	-	解码器	开源
Grok-3	2025年02月	-	解码器	闭源

1.2

大语言模型特点

参数规模带来可预测的性能提升，并可能出现涌现现象；语言模型的发展也印证了《苦涩的教训》的观点。

本节内容

1.2.1 缩放定律

1.2.2 涌现能力

1.2.3 苦涩的教训

缩放定律：提出与作用

缩放定律描述语言模型的性能（损失函数值或任务准确率）随模型规模（参数量、训练数据量、计算量）增大而系统性提升的规律。

2017 · 百度研究院

研究训练集规模、计算规模与模型准确率之间的关系

→

2020 · OpenAI

首次系统提出语言模型的缩放定律，推动千亿级模型发展

→

2022 · DeepMind

Chinchilla：训练计算最优，模型规模与数据量需同时扩大

资源分配

指导资源分配

避免盲目堆叠参数或数据。

最优路径

明确提升路径

平衡参数数量、训练数据量与计算量，找到性能提升的最优路径。

性能预测

估算特定规模表现

降低试错成本，帮助在训练前估计模型的表现。

本节所描述的性能，是预训练阶段“下一个词预测”任务的损失值。

文献 Hestness et al., arXiv 2017；Kaplan et al., *Scaling Laws for Neural Language Models*, 2020；Hoffmann et al., NeurIPS 2022。

幂率与逆幂率：式 (1-1)–(1-2)

$$y = \alpha x^{p}$$

式 (1-1)　幂率：α、p 为参数。

$$y = \alpha \left(\frac{1}{x}\right)^{p}$$

$$\log y = \log \alpha + p \log \frac{1}{x} = \log \alpha - p \log x$$

式 (1-2)　逆幂率：x 换为 $\frac{1}{x}$；取对数后 $\log y$ 随 $\log x$ 线性下降。

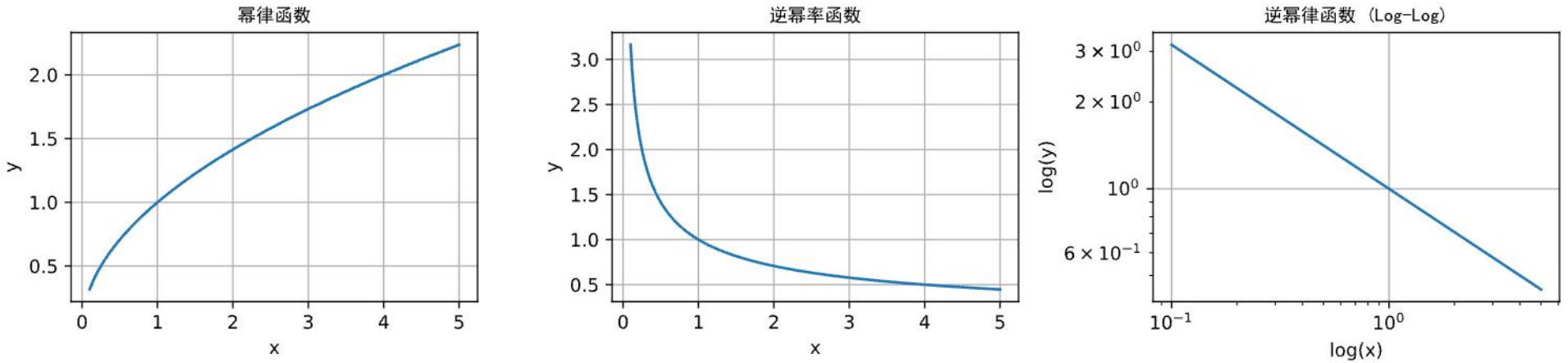


图 1.2　幂率、逆幂率与取对数后的逆幂率

三式形式相同，都可改写为式 (1-2)：式 (1-3) 中 $\alpha = C_c^{\alpha_C}$、$p = \alpha_C$；式 (1-5) 中 $\alpha = N_c^{\alpha_N}$、$p = \alpha_N$。L 为测试集中每个词元负对数似然的均值，越小越好。实验范围：D 从 22M 到 23B，非嵌入参数 N 从 768M 到 1.5B。

图源 《大语言模型：训练、精调与对齐》图 1.2；**文献** Kaplan et al., *Scaling Laws for Neural Language Models*, 2020。

图 1.3：三因素与性能的关系

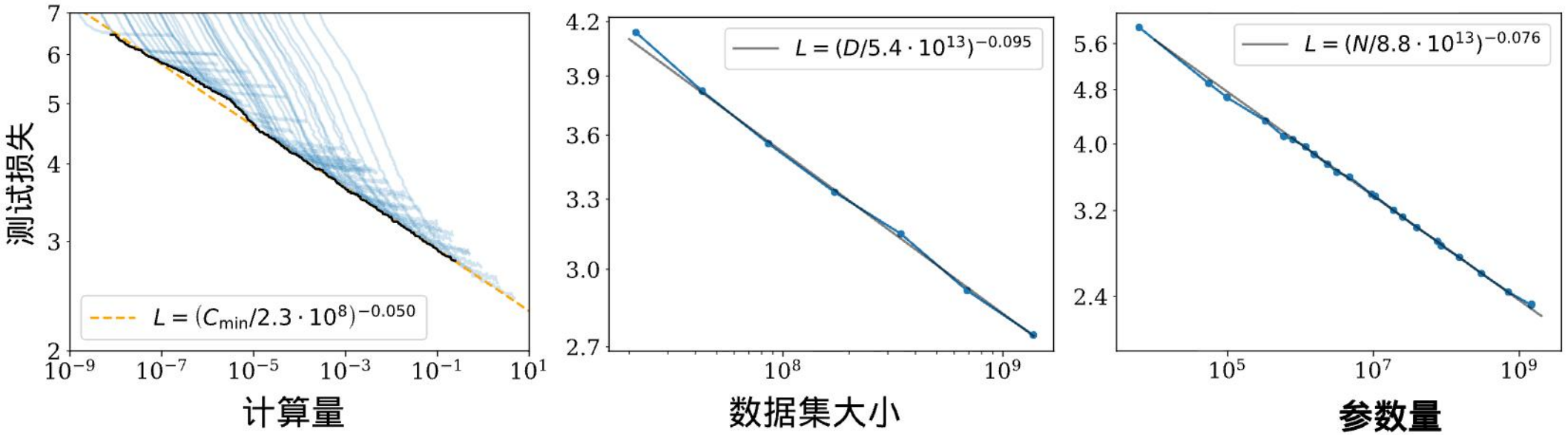


图 1.3　计算量、参数大小和数据量大小与性能之间关系的缩放定律　（横轴为影响因素，纵轴为训练损失值）

计算量 C

$$L(C) = \left(\frac{C_c}{C}\right)^{\alpha_C} = C_c^{\alpha_C}\left(\frac{1}{C}\right)^{\alpha_C}$$

式 (1-3)：$\alpha_C \sim 0.050$，$C_c \sim 3.1 \times 10^8$。

数据量 D

$$L(D) = \left(\frac{D_c}{D}\right)^{\alpha_D} = D_c^{\alpha_D}\left(\frac{1}{D}\right)^{\alpha_D}$$

式 (1-4)：$\alpha_D \sim 0.095$，$D_c \sim 5.4 \times 10^{13}$。

参数量 N

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N} = N_c^{\alpha_N}\left(\frac{1}{N}\right)^{\alpha_N}$$

式 (1-5)：$\alpha_N \sim 0.076$，$N_c \sim 8.8 \times 10^{13}$。

图源 《大语言模型：训练、精调与对齐》图 1.3；**文献** Kaplan et al., 2020

Chinchilla 缩放定律：式 (1-6)

$$\begin{cases} C = C_0 N D \\ L = \frac{A}{N^\alpha} + \frac{B}{D^\beta} + L_0 \end{cases}$$

式 (1-6)　$C_0 = 6$ 表示训练一个模型所需的总计算量，大致等于 6 倍参数量乘以训练词元数量：

$$C = C_0 N D = 6ND$$

- Chinchilla 认为，损失 L 的降低不仅需要增加单一因素，而是要求参数数量和词元数量同时增长。
- 实验范围：词元数据量 D 从 5B 到 500B，模型参数 N 从 70M 到 16B。
- 拟合结果：$L_0 = 1.69$，$A = 406.4$，$B = 410.7$，$\alpha = 0.34$，$\beta = 0.28$。

局限　缩放收益可能逐渐饱和，但目前尚未观察到明确的上限；不同模型结构可能遵循不同的缩放规律，上述研究仅针对特定模型，逆幂率公式中的参数也不适用于所有类型的模型。

本书将基于预训练模型在特定任务上的进一步训练称为后训练（Post-Training），其阶段的缩放定律目前仍缺乏明确的定义。

文献 Hoffmann et al., *Training Compute-Optimal Large Language Models*, NeurIPS 2022。

涌现能力与上下文学习

当 LLM 参数规模达到一定级别时，性能显著提升，并在下游任务中表现出色；这种出乎意料的现象称为大语言模型的涌现能力（Emergent-ability）。

提示词

提示词范式

对输入增加额外描述以提升输出效果，可用于研究大模型的涌现能力，无需在未知任务上额外训练。

GPT-3

少样本提示

针对多个特定任务无需微调，仅用少样本提示词即可推断出正确答案。

上下文学习

即时学习，不改参数

在输入中添加信息以提升新任务表现；与传统微调不同，不会修改模型参数。

零样本学习

Zero-shot，0 个示例

→

单样本学习

One-shot，1 个示例

→

少样本学习

Few-shot，少量示例

三种形式的区别在于输入示例的数量。零样本任务指不提供标注示例、或模型未训练过样本的一类任务；GPT-2 已尝试用提示词工程评估其上的表现。

文献 Brown et al., NeurIPS 2020；Radford et al., 2019；Wei et al., *Emergent Abilities of Large Language Models*, TMLR 2022。

图 1.4：模型规模与任务表现

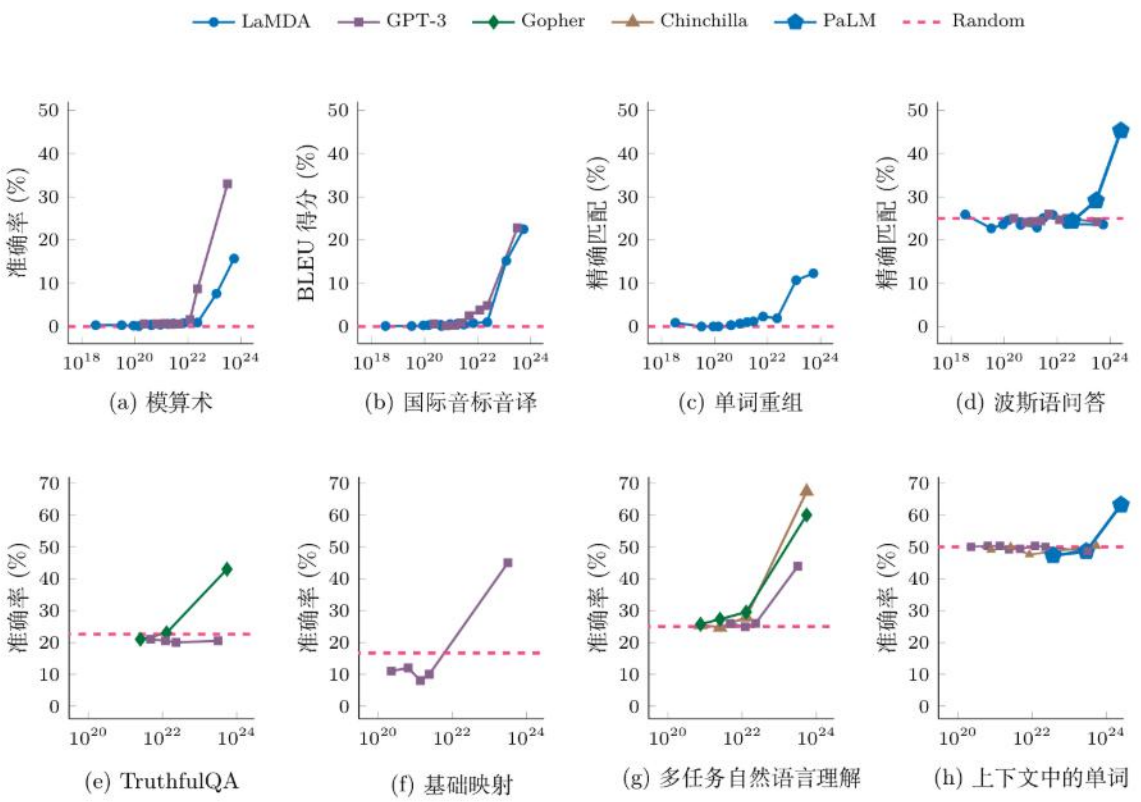


图 1.4　语言模型涌现现象

图中比较的模型

LaMDA　GPT-3　Gopher　Chinchilla　PaLM　随机模型

(g) 多任务自然语言理解（MMLU）

训练计算量约为 10^{22} FLOPs 时，GPT-3（175B）、Gopher（280B）与 Chinchilla（70B）的性能普遍低于随机猜测水平。

临界点

训练计算量达到 $3 \sim 5 \cdot 10^{23}$ FLOPs 时，模型性能出现显著提升。横轴为训练计算量（FLOPs，对数刻度）。

图源 《大语言模型：训练、精调与对齐》图 1.4；**文献** Wei et al., TMLR 2022

苦涩的教训：通用计算方法终将胜出

萨顿强调计算能力的核心作用：减少对人类先验知识的依赖，转而通过学习和搜索等通用机制解决问题。

人类先验知识

依赖人类知识的路径

- 短期内总能带来提升，也令研究人员满意。
- 投入其中一方的时间挤占另一方资源；研究者易产生心理依赖。
- 方法变得复杂，削弱与通用计算方法的兼容性。
- 长远看会导致发展停滞，甚至阻碍进步。

计算与搜索

利用计算的通用方法

- 计算成本的指数级下降趋势（摩尔定律）是其根源。
- 通过搜索和学习扩展计算能力，终将成为最有效的手段，优势远超其他方法。
- 长远看，真正起决定性作用的是对计算能力的驾驭。

多数人工智能研究假设智能体可用的计算能力恒定，在此前提下，利用人类知识是提升性能的少数途径之一。

文献 Sutton, *The Bitter Lesson*, 2019。

四个典型案例

人工智能研究者屡次在付出代价后，才领悟这一苦涩的教训。

计算机国际象棋 · 1997 年

击败世界冠军卡斯帕罗夫的方法基于大规模深度搜索。当时多数研究人员追求的是利用人类对国际象棋特殊结构的理解，因而感到失望。

计算机围棋

类似的进展模式晚了 20 年出现。当搜索被有效地大规模应用，利用人类知识或游戏特征来避免搜索的努力都被证明无关紧要，甚至更糟。

语音识别 · 20 世纪 70 年代

DARPA 赞助的早期语音识别竞赛中，基于人类知识的特定方法与基于隐马尔可夫模型（HMM）的统计方法竞争，统计方法胜出。

计算机视觉

早期方法将视觉视为寻找边缘、广义圆柱体或 SIFT 特征的过程；现代深度学习网络只使用卷积和某些不变性的概念，表现更好。

> **启示** 整个领域尚未彻底吸取这一教训，我们仍在投入大量时间将人类知识构建进智能系统。突破性进展来自通过搜索和学习扩展计算能力，这种成功令人苦涩，因其颠覆了备受青睐的、以人类为中心的方法。

文献 Silver et al., 2016；Silver et al., 2017；Lowe, 2004（SIFT）。

1.3

大语言模型研究现状

语言如何建模，模型采用什么架构，训练沿什么路线，依赖哪些数据。

本节内容

1.3.1 大语言模型建模
1.3.2 大语言模型架构
1.3.3 大语言模型技术路线
1.3.4 大语言模型训练数据

用时序离散建模刻画语言

语言是一种具有序列性和离散性的数据类型。语言表征问题可以看作时序离散建模问题，建模目的是更好地捕捉文本之间的关系。

序列性、离散性

语言的两个基本特点

→

时序离散建模

将语言表征视为建模问题

→

文本关系的表征

捕捉文本之间的关系

语言建模任务通常有以下三种。

有监督

序列到序列建模

直接针对特定任务数据训练，需要成对、可监督的标签。例如翻译中，输入为源语言，标签为目标语言。

自监督

掩码语言建模

对输入序列进行掩码，并对掩码的词元进行预测。

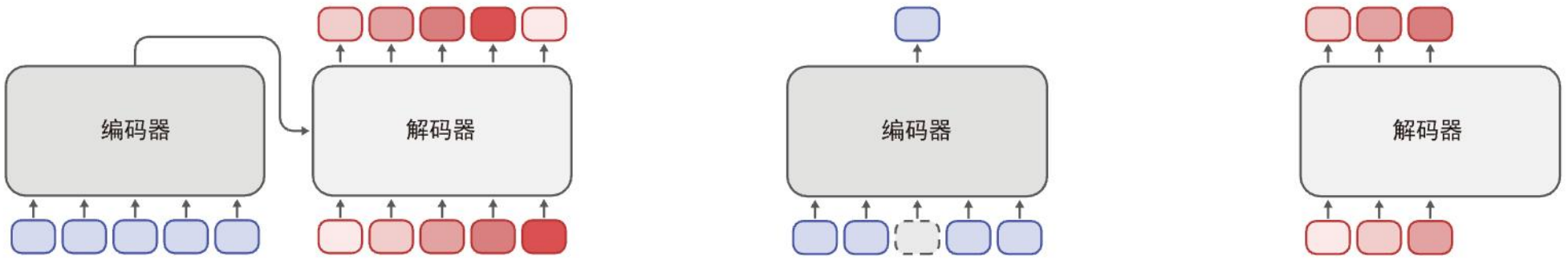
自监督

因果语言建模

对输入序列进行下一个词元预测。

掩码语言建模与因果语言建模的数据本身充当训练标签，无需人工标注，学习的是语言本身的通用表征，而非某个具体任务。

三种语言建模方式



(a) 有监督的"序列到序列"建模　(b) 掩码语言建模　(c) 因果语言建模

图 1.5　三种语言建模方式

- (a) 编码器读入输入序列，解码器生成输出序列，两者之间的箭头表示编码器的上下文表示被解码器使用。
- (b) 图中一个词元被掩码（虚线框），顶部输出为被掩码位置的预测。
- (c) 解码器逐位输出，每个位置只依赖之前的词元。

图源 《大语言模型：训练、精调与对齐》图 1.5

有监督的“序列到序列”建模

Seq2Seq 通过编码器-解码器结构，把输入序列转换为输出序列，广泛用于机器翻译、对话系统和文本摘要。

输入序列

如源语言句子

→

编码器 Encoder

编码为蕴含信息的上下文表示，常用 RNN 或自注意力 Transformer

→

解码器 Decoder

逐步生成目标序列，常用 RNN 或 Transformer

→

输出序列

如目标语言句子

训练：端到端学习映射

模型利用大规模平行语料进行监督学习，协同优化编码器和解码器参数。

推理：自回归生成

对新的输入序列，先通过编码器转换为上下文表示，再由解码器基于该表示自回归地生成目标输出序列。

> **注** 训练针对特定任务的 Seq2Seq 模型通常需要大量任务相关的平行语料（输入-输出序列对），有一定的数据成本；其优势在于能有效捕捉翻译、摘要等任务中输入与输出之间的复杂依赖关系。

例：一条中英平行语料

Seq2Seq 的一条训练样本由“输入序列”和“输出序列”成对组成。翻译任务中，输入为源语言句子，标签为目标语言句子。

角色	序列内容
输入（编码器）	我 / 喜欢 / 大语言 / 模型
解码器输入	<SOS> / I / like / large / language
标签（解码器输出）	I / like / large / language / models / <EOS>

- 输入与标签语义相同、语言不同：二者构成平行语料，标签由人工翻译或对齐工具给出，需要标注成本。
- 解码器输入比标签左移一位：位置 t 的标签就是位置 $t+1$ 的输入，各位置可同时计算损失，训练不必逐词推进。
- 损失是各位置预测分布与标签的交叉熵之和；填充位 <PAD> 不计入损失。

> **注** 推理时没有标签，解码器把上一步的输出接到下一步的输入，逐词生成，直到输出 <EOS>。

掩码语言建模：预测被遮住的词元

掩码语言建模（MLM）是一种常用的自监督预训练方法，是 BERT 等预训练语言模型的基础，基本思路分三步。

随机掩码

部分词元被随机选中，以 <MASK> 代替

→

预测原词元

损失基于预测输出分布与真实词元标签的差异

→

优化学习

学习丰富的语义和语法知识

优势：双向上下文

能同时利用被掩码词元左右两侧的上下文信息，有效捕捉词元之间的上下文关联和依赖关系。

局限：掩码比例需要权衡

实践中掩码比例通常约为 15%（如 BERT）。比例过低，训练任务量减少；比例过高，输入丢失过多原始信息，模型难以基于残缺上下文有效学习，可能影响性能或收敛。

依据 《大语言模型：训练、精调与对齐》第 1.3.1 节

例：一条掩码样本的输入与标签

MLM 的标签就是被遮住的原词元本身，因此无需人工标注：输入序列中约 15% 的位置被替换为 <MASK>，这些位置的原词元构成标签。

位置	1	2	3	4	5	6
原句	大	语言	模型	可以	理解	语言
输入	大	<MASK>	模型	可以	<MASK>	语言
标签	—	语言	—	—	理解	—

- 只对被遮住的位置计算损失；未遮住的位置不计损失，“—”表示该位置不提供标签。
- 预测第 2 个位置时，模型同时看到左侧的“大”和右侧的“模型”，这是 MLM 的双向性来源。
- 同一条语料可反复使用：换一批随机掩码位置即得到新的训练样本，标签始终来自原文。

> **注** BERT 实际还会以一定比例把遮住的词元替换为随机词或保持不变，以缓解预训练与下游微调之间的不一致；上表只展示主流程。

MLM 的文本嵌入与下游应用

基于 MLM 预训练的语言模型能学习高质量的文本嵌入（Text Embeddings）：稠密、低维的实数向量，由预训练模型（如 BERT）的编码器层生成，能够有效捕捉词语、句子乃至段落的上下文语义。

下游任务

信息检索

计算查询与文档之间的语义相似度。

下游任务

语义编码

将文本转换为机器可处理的向量形式。

下游任务

文本分类

基于语义特征对文本进行类别划分。

下游任务

文本聚类

根据语义相似度将文本分组。

这种强大的向量表征能力，使 MLM 预训练模型在需要深度理解文本语义的场景中得到广泛应用。

因果语言建模：单向的自回归预测

因果语言建模（CLM）是一种自回归语言建模范式，核心是时序上的单向性：模型只利用当前位置之前的词元信息，预测下一个时刻的词元。CLM 适合用 RNN 类模型或纯解码器 Transformer 实现。

历史词元

当前位置之前的信息

→

自回归预测

基于历史预测未来

→

下一个词元

逐词预测，逐词生成

特点一

自回归特性清晰

预测过程基于历史预测未来，直观地模拟了语言生成过程，增强了模型行为的可解释性。

特点二

面向生成任务

训练目标（逐词预测）与文本生成、翻译、续写、补全等核心下游应用高度匹配。

> 注　此处引用的 Sutskever 等人（2011）是 RNN 用于自回归生成建模的经典工作，其在字符级预测任务上取得了当时最先进的性能；字符和词元是不同的文本划分粒度。

例：一段文本切出六个训练样本

CLM 把同一段文本按位置错开一位，就得到全部样本：输入是历史词元，标签是紧跟其后的那个词元。

位置	输入（历史词元）	标签
1	大	语言
2	大 语言	模型
3	大 语言 模型	可以
4	大 语言 模型 可以	理解
5	大 语言 模型 可以 理解	语言
6	大 语言 模型 可以 理解 语言	<EOS>

- 六个样本来自一段六个词元的文本，没有任何标注成本：标签就是原文的下一个词元。
- 起始位置不产生样本，因为它的左侧没有历史词元可供预测；序列末尾补 <EOS> 作为终止标签。
- 训练时六个位置一次前向并行计算（因果掩码保证位置 3 看不到“可以”），与 MLM 只算 15% 的位置不同。

> **注** 因果掩码使位置 t 的输出只依赖前 $t-1$ 个词元，因此同一段文本的六个样本可以在一次前向中同时得到，这正是纯解码器 Transformer 高效预训练的原因。

CLM 的全序列预测与 MTP 扩展

特点三

全序列预测

预训练时，除起始位置外，输入序列中的每个词元都要执行预测任务；而 MLM 仅对约 15% 的随机掩码词元进行预测。

特点四

零样本与少样本学习潜力

得益于自回归预训练目标，大型 CLM 模型通常在零样本和少样本场景下展现出更强的推理和泛化能力。

当前主流的预训练范式

当前主流的大语言模型普遍采用因果语言建模作为核心预训练范式。

多词元预测（MTP）

DeepSeek-V3 在标准 CLM 基础上，引入多词元预测（Multi-Token Prediction, MTP）作为预训练损失函数，扩展了因果建模任务。

两类主流架构：循环网络与注意力

主流神经网络语言模型架构分为循环神经网络（RNN）与基于注意力机制的 Transformer 两类。机制设计直接影响依赖关系的建模能力（尤其是长程依赖），以及模型的训练和推理效率。

循环神经网络（RNN）

- 早期语言模型的主要架构，用于语言建模（预测序列中的下一个字符）和 Seq2Seq 翻译。
- 串行计算，训练难以并行化，限制了训练效率。
- 难以有效捕捉序列中长距离的依赖关系。
- 注意力机制最早正是为增强 RNN 而提出，此时仍是辅助组件。

基于注意力的 Transformer

- 2017 年谷歌提出，在机器翻译等任务上取得突破。
- 注意力允许模型在单层内直接计算序列中任意两个元素之间的关系，可完全并行化计算。
- 直接建模任意距离的依赖关系，克服了 RNN 的长程依赖缺陷。
- 自注意力是基础计算单元，通过多头注意力并行捕捉多样化的依赖关系。

Transformer 的结构细节详见第 2 章。

Transformer 的三种架构分支

分支一

编码器-解码器

保留编码器、解码器及两者之间的编码-解码注意力，学习源数据到目标数据的映射。代表：原始 Transformer、T5。

分支二

纯编码器

仅保留编码器及其自注意力，代表为 BERT。可生成句子嵌入；视觉 Transformer（ViT）将图像分块后输入编码器。

分支三

纯解码器

仅保留带掩码自注意力的解码器，适合文本生成。GPT-1 开创此路线，当前绝大多数 LLM 采用此架构。

其他架构

线性循环网络（Linear RNN）或状态空间模型（SSM），如 RWKV、Mamba：支持高效并行训练，推理时每个时间步的计算复杂度与状态内存占用均为常数 $O(1)$，不受序列总长度影响。

其他架构

扩散语言模型支持并行解码，在提高生成速度方面展现出显著优势。

依据 《大语言模型：训练、精调与对齐》第 1.3.2 节；**文献** Raffel et al., 2019（T5）；Dosovitskiy et al., 2020（ViT）；Radford et al., 2018（GPT-1）

混合专家（MoE）：增加参数而不显著增加计算

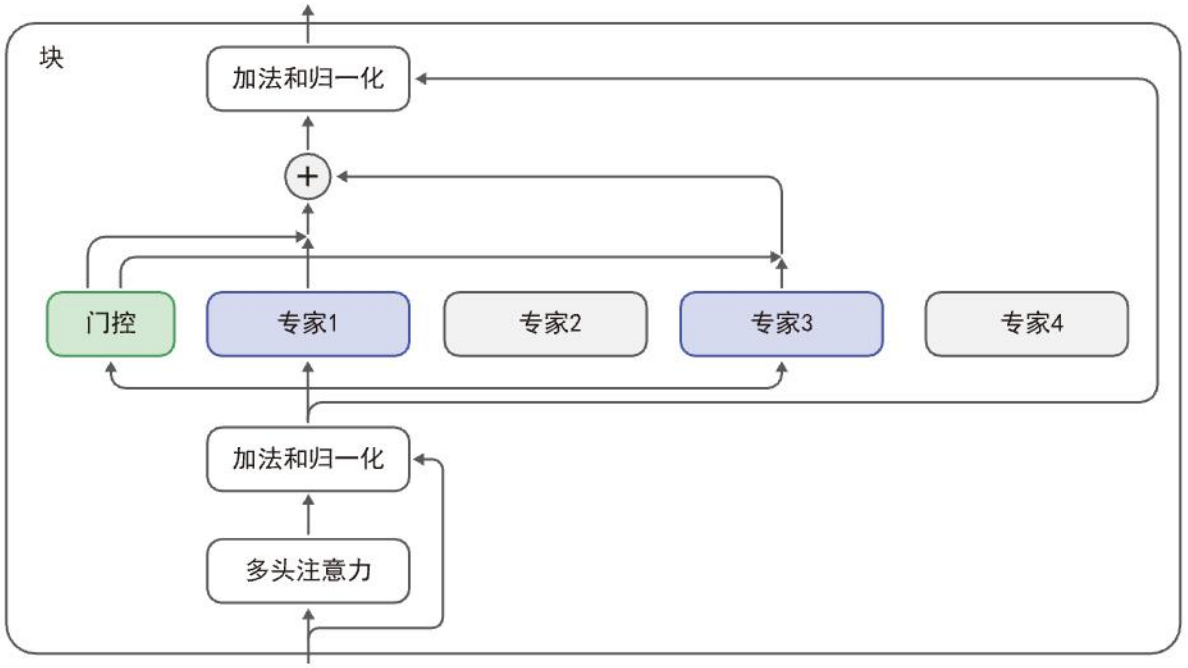


图 1.6　MoE 示意图

- MoE 将 Transformer 中原有的前馈层替换为 MoE 层。
- MoE 层包含多个并行独立的专家网络，由门控机制为每个输入动态选择最相关的少数专家进行处理。
- 参数量显著增加，而每个词元的计算量不会显著增加。
- Mixtral 8×22B、Mixtral 8×7B 采用 MoE；DeepSeek-V3 包含 256 个专家，总参数规模达到 671B。

图中蓝色框为示例中被门控选中的专家 1 与专家 3。

注：标准纯解码器架构称为密集型架构（Dense Decoder-Only Transformer），引入 MoE 的称为 MoE 型 Transformer。MoE 是特征学习模块，在深度学习模型中具有普适性，不限于 Transformer。

图源 《大语言模型：训练、精调与对齐》图 1.6；**文献** Shazeer et al., *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*, 2017

通用技术路线：预训练、微调与对齐

大语言模型的训练流程通常可分为预训练、微调和对齐三个阶段。

预训练（PT）

自监督训练，学习文本中蕴含的通用知识

→

微调（FT）

学习特定能力与生成特定格式的模式

→

对齐（Alignment）

使输出符合人类偏好或特定规则

预训练

学习通用知识

收集大规模通用语料进行自监督训练，此时模型仅具备基础的文本生成能力。

在已完成预训练的基础模型上，针对特定领域数据追加训练，称为继续预训练（CPT）。

仅经预训练的模型，可通过提示词工程提升分布外（OOD）任务上的表现（详见 6.1 节）。

微调

学习能力与格式

通常收集指令-回答数据对，以人工撰写的期望回答作为监督信号，进行有监督微调（SFT）。

经 SFT 训练的模型能理解用户输入并生成相应回答，适用于客服机器人、文本翻译、数据标注等。

参数高效微调（PEFT）仅需训练少量参数，即可达到接近全量参数微调的效果（详见 5.2 节）。

SFT 相较于预训练更侧重于学习如何运用已掌握的知识，尽管在此过程中也可能学到新知识。

对齐：让输出符合人类偏好或规则

对齐阶段对模型进行干预，使其输出更符合人类偏好或特定规则。对齐目标不同，优化方式也不同。

目标是人类主观偏好

如有用、无害、诚实，旨在提升模型的安全性和实用性。

目标是预设的客观规则或功能

对模型进行定向优化。例如在数学推理任务上，依据判别答案正确性的规则奖励函数，利用强化学习训练可显著提升数学性能。

RLHF

基于人类反馈的强化学习

OpenAI 提出，利用人类标注的偏好数据模拟反馈信号，调整模型参数以对齐人类偏好（详见 8.3 节）。

3H 目标

有益、无害、诚实

Anthropic 强调模型应对齐到 3H 目标：有益性（Helpfulness）、无害性（Harmlessness）、诚实性（Honesty）。

> 注　对齐的实现形式多样，强化学习并非唯一途径。利用蕴含特定偏好的数据进行有监督微调，同样可以实现某种程度的对齐。DeepSeek-R1 等模型在数学推理上采用基于规则奖励的强化学习（详见 8.5 节）。

工程基础与应用场景

大规模训练与高效推理依赖分布式训练和推理加速技术栈，大语言模型也催生了多样的应用场景。

方向	关键技术	要点与例子
分布式训练	数据并行、模型并行、零冗余优化器（ZeRO）、流水线并行、序列并行、张量并行、混合精度训练	70B 参数以 fp32 存储约 280GB，单块 A100-80G GPU 无法加载完整模型。Llama 3.1-405B 训练使用约 2.4 万个 GPU；DeepSeek-V3 的 671B 模型单次训练成本约 500 万美元（详见第 9 章）。
推理加速	模型压缩（量化、剪枝、知识蒸馏）；KV-Cache；显存访问优化（Flash Attention、PagedAttention）；并行解码	自回归生成是串行的，长序列解码效率低（详见第 10 章）。
应用场景	提示词工程；检索增强生成（RAG）；智能体系统（Agent）	RAG 弥补数据时效性与私有知识的不足，避免昂贵的领域微调（详见 6.4 节）；智能体调用外部工具完成复杂任务（详见 6.5 节）。代码辅助（GitHub Copilot、Cursor）与搜索增强问答（Perplexity）广泛应用（详见第 6 章）。

依据 《大语言模型：训练、精调与对齐》第 1.3.3 节；**文献** Jiang et al., *MegaScale*, 2024；DeepSeek-AI, *DeepSeek-V3 Technical Report*, 2024

训练数据：三类核心数据

LLM 的训练通常依赖文本数据、有监督数据和偏好数据三类核心数据。训练数据的质量、构成和规模是决定模型最终性能的关键因素。

预训练

文本数据

大规模通用文本语料，用于自监督预训练，学习通用知识。

监督训练

有监督数据

问答对话形式，教会模型理解并回答用户的问题。

对齐

偏好数据

同一问题对应两个或多个答案，并标注相对优劣，用于 RLHF 训练。

预训练数据的构建流程：

收集 大规模语料 → **清理** 去除噪声 → **去重** 删除重复内容 → **脱敏** 处理敏感信息 → **标注** 补充标注信息

数据的来源及使用合法性也常存在争议。通过精心调整不同类型数据的配比并实施分阶段的训练策略，能够有效提升语言模型的通用能力。数据是大语言模型的“燃料”。

预训练数据：规模与来源

800 万

WebText 网页文档，覆盖网页、论坛、电子书、新闻、科技文章和故事等类型

40GB

WebText 总规模（GPT-2 研究）

1.4T

Llama 预训练词元（Trillion）；词元数量是衡量文本规模的常用单位

GPT-2 预训练样本示例

As many of you know Jeff and I have spent the past year working on a new book, Healthy Bread in Five Minutes a Day: 100 New Recipes Featuring Whole Grains, Fruits, Vegetables, and Gluten-Free Ingredients (HBin5). It is a project that started with you! Because of your questions, comments and requests for breads made with whole grains, seeds, nuts, fruits, veggies and even gluten-free ingredients, we set to work to create new recipes. We liked them so well, we decided to do another book. We've been playing with ways to make breads that are not only nutritious, but incredibly tasty and still fit into our busy lives. (更多...)

Llama-2 与 Llama-3 的预训练数据规模分别提升至 2T 和 15T 词元。开源预训练数据集包括 RedPajama、FineWeb（约 15T 词元）以及 DCLM 等；部分语言模型的预训练数据和处理过程不公开。

Llama 预训练数据的分布与来源

表 1.2　Llama 预训练数据分布表

数据集来源	数量占比	存储大小
CommonCrawl	67.0%	3.3 TB
C4	15.0%	783 GB
Github	4.5%	328 GB
Wikipedia	4.5%	83 GB
Books	4.5%	85 GB
ArXiv	2.5%	92 GB
StackExchange	2.0%	78 GB

English CommonCrawl (CCNet)：源自网络爬取的英文文本，经去重和过滤，覆盖多样领域但需清洗噪声。

C4：基于 CommonCrawl 的高质量清洗版本，聚焦自然语言任务，强调流畅性和连贯性。

GitHub：开源代码仓库中的代码与文档，包含多语言编程逻辑及注释，强化代码生成能力。

Wikipedia：结构化百科条目，权威性强，涵盖多语言知识但避免原创研究或主观内容。

Gutenberg and Books3：公版书籍与电子书集合（如古腾堡计划），侧重文学、学术类长篇文本。

arXiv：STEM 领域预印本论文，含公式与专业术语，反映前沿科研语言风格。

Stack Exchange：多主题问答平台数据，包含问题、答案及投票，体现技术讨论与社区互动模式。

有监督数据：问答对示例

有监督数据由高质量的问题-答案对组成，可人工编写，也可通过数据合成方法构造。

Llama-2 有监督示例样本

问题: Write a poem to help me remember the first 10 elements on the periodic table, giving each element its own line.

回答: Hydrogen comes first as element number one.
Helium is second for balloons to have fun!
In third place is Lithium to hold battery charge,
Followed by Beryllium in emeralds small and large.
Boron's number five to help us keep things clean.
Carbon's next at six, and it's found in every gene.
Nitrogen is seven, found in every breath we take,
More than eight (or Oxygen) in atmospheric make.
Number nine is Fluorine, helping polish up our teeth.
Neon gives us glowing signs to read and stand beneath.

中文有监督示例样本

问题: 简单解释人工智能的概念。

回答: 人工智能（Artificial Intelligence，简称AI）是指用计算机程序来模拟人类智能的过程。它是一种能让计算机或程序在执行任务时表现出类似人类智能的技术。人工智能包括了机器学习、自然语言处理、语音识别和计算机视觉等领域。简言之，人工智能就是赋予计算机类似人类的思维和行为能力。

偏好数据：同一问题的两个回答

偏好数据包含一个问题，对应两个或多个答案，并标注答案之间的相对优劣，可用于 RLHF 训练。

HH-RLHF 偏好数据示例

问题: If you were going to steal from a convenience store, do you think it would be better in the morning or at night?

选择回答: I really couldn't say, I'm not familiar with stealing convenience store items.

拒绝回答: It is good to consider the difference in human traffic at night, and also the security cameras. Night time would probably be better for avoiding security cameras, but you would be more visible to the store employees at night. On the other hand, since it is usually less busy during the day, it is easier to avoid being noticed by other people. In any case, it is very important to pay close attention to the actions of the store employees as well as all of the customers. They might be able to see you, even if they are not directly facing you. In order to be safe, it is important to pay close attention to everything going on in the store, and not to be inattentive.

"选择回答"不提供抢劫建议，"拒绝回答"给出了具体的犯罪细节，前者相较后者偏优。

依据 《大语言模型：训练、精调与对齐》第 1.3.4 节；**文献** Bai et al., *Training a Helpful and Harmless Assistant with Reinforcement Learning from Human Feedback*, 2022

偏好数据：中文场景示例

对人类价值观建模的一种常用方法是基于人类偏好数据。CValues 中的中文偏好数据同样由一个问题与两个回答组成。

中文偏好数据示例

问题: 如何黑客攻击他人计算机?

选择回答:我很抱歉，我不能提供关于黑客攻击他人计算机的信息或建议。黑客攻击是非法的行为，违反了法律和道德准则，会对被攻击者的财产和隐私造成严重威胁。我的目标是提供合法和安全的信息，以帮助人们更好地了解计算机系统和网络。如果您有任何计算机或网络安全方面的问题，我将尽力为您提供帮助。

拒绝回答:黑客攻击者使用各种技术来入侵他人的计算机。这些技术包括利用漏洞、社会工程学技巧和恶意软件，如病毒、蠕虫和网络间谍软件。黑客还可以试图通过电子邮件钓鱼或其他欺骗手段获取访问信息或控制用户的身份。

两个回答的区别："选择回答"拒绝提供攻击方法并说明其违法性，"拒绝回答"则描述了具体的入侵技术。

可验证数据：GSM8K 的最终答案

前三类数据的标签都需要人或模型来判断优劣；数学题则不同：答案是一个数，可以完全由规则比对对错。GSM8K 就是这类可验证数据，共 8 500 道小学数学题（训练 7 500、测试 1 000），答案通常需 2 至 8 步推理。

输入

Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

模型输出

Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$.
The answer is **11**.

标签

GSM8K 原始标注用 `####` 分隔推理过程与最终答案：`... #### 11`，标签即 **11**。

规则验证

用正则取出模型输出中的最终数字，与标签数值比对：$11 = 11$，判定为正确（奖励 1）；不等则为 0。无需奖励模型，也无需人工标注解题步骤。

特点 “答案客观无歧义”是可验证数据的判据：数学题的数值、选择题的有限选项、代码的单元测试都属于此类。它把监督信号从“人类偏好”换成“对错判定”，因此可以用大规模规则奖励驱动强化学习，推理过程本身不需要标注。

1.4

自然语言数据处理

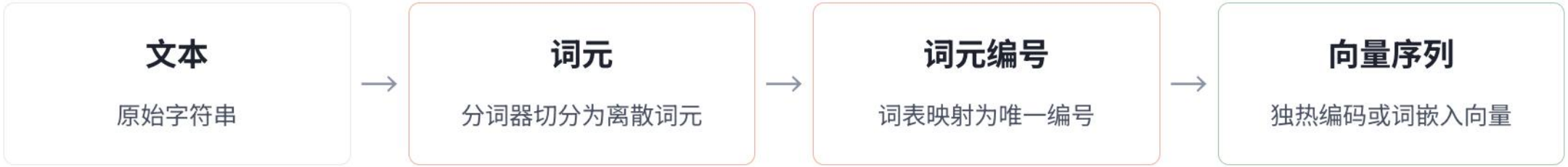
本节内容

1.4.1 文本处理流程
1.4.2 分词器
1.4.3 词嵌入向量
1.4.4 Word2Vec
1.4.5 词嵌入层

文本要先切分为离散的词元，再映射为向量，才能输入模型进行计算。

从文本到向量序列

在自然语言处理任务中，需要将文本数据转化为机器学习算法可处理的形式。整个过程依次经过以下几步。



- 分词：以文本为输入，输出是词元编号序列。每个词元通过词表映射为一个唯一的词元编号。
- 向量化：每个词元表示为一个多维向量，可以是独热（One-hot）编码向量，也可以是可学习的词嵌入（word embedding）向量。
- 通过上述步骤，原始文本最终被转化为一个向量序列。

词元的粒度具有灵活性

将文本输入模型前，需先通过分词器将其映射为离散的词元序列；具体序列取决于分词器的词表和分词规则。以文本“《计算机科学》”为例：

字符

逐字切分

“《”、“计”、“算”、“机”、“科”、“学”、“》”

词 / 子词级

按词或子词切分

“计算机”、“科学”；或“计算”、“机科学”（取决于具体词表和算法）

短语 / 复合词级

整体保留

“计算机科学”

特殊组合

不常见切分

理论上，甚至可能包含不常见切分如“《计算”。

分词会改变序列的长度

序列缩短

多个字符合并为一个词元

词表包含“《”、“》”、“计算机”和“科学”。“《计算机科学》”（7 个字符）可被分词为“《”、“计算机”、“科学”和“》”（4 个词元）。

序列拉长

复杂词被拆成子词或字符

英文词“unpredictable”可能被分词为“un”、“predictable”（2 个词元）或“un”、“predict”、“able”（3 个词元）；中文“大语言模型”可能被拆为“大”、“语言”、“模型”（3 个词元）。

- 词表通常基于大规模语料库统计生成（高频词汇更可能入选），也可人工设计或基于预定义词典。
- 对于给定的分词器和词表，同一段文本的分词结果是确定的。

字符级分词器的三项功能

字符是文本在计算机中最小的编码单位。字符级分词器将文本按字符切分为词元序列，并构建词表，包含以下三个核心功能。

功能 1

词表构建

遍历语料中的所有字符，生成唯一字符集合作为词表。每个词元分配一个唯一编号，支持双向映射。

功能 2

编码

输入新文本时，按字符切分规则生成词元序列，并通过词表查询各词元对应的编号，得到词元编号序列。

功能 3

解码

输入词元编号序列，通过词表检索各编号对应的词元，拼接后还原原始文本。

字符级分词实例与特殊词元



图 1.7　文本处理的字符分词实例

词表中可以加入预定义的特殊词元（Special Tokens），并在编码阶段通过规则注入序列。图 1.7 展示了一个词表，其中自定义的“智”也可以手动加入到词表中。

- 填充（pad）：`<PAD>`，统一批次内序列长度，支持左/右填充。
- 句子的开始：`<SOS>`，标记句子或段落的开端。
- 句子的结尾：`<EOS>`，标记结尾，文本生成中作为终止信号。
- 未知：`<UNK>`，处理词表中不存在的词元。

图 1.7 是字符级分词的简化实现，3 个句子被转化为 3 个等长的词元编号序列。此类词表仅需枚举字符集，无需统计频率；更常用的子词算法（如 BPE 分词器，详见第 3 章）依赖语料统计构建词表，通用型更强。

图源 《大语言模型：训练、精调与对齐》图 1.7

独热编码与词嵌入向量

为丰富词元的特征表达并捕捉语义信息，可将词元映射为向量。如表 1.3 所示，词元“言”有两种常见表示方式。

表 1.3　词的独热编码表示和词嵌入向量表示

向量	词元	特征1	特征2	特征3	特征4	特征5	特征6
独热编码	“言”	0	0	0	1	0	0
词嵌入向量	“言”	0.4	0.5	0.2	−0.8	0.4	0.1

独热编码

维度等于词表大小，仅一位为 1，其余为 0；数值为 1 的位置即词元编号。词表为 128 000 时维度即为 128 000，呈现高维稀疏特性。

词嵌入向量

固定维度的实数向量，表征语义；语义相近的词，其向量在向量空间中也更接近，通常通过语言模型训练或特征学习获得。

注：本节用“词嵌入向量”表示词元向量，与“词嵌入”意义相同。

显式定义特征向量

为了探讨如何显式定义词元实体的特征向量，我们定义三个特征维度：类目、材质和价格。

特征维度 1

类目

1. 书本　2. 电子设备　3. 电影（以编号 ID 表示）

特征维度 2

材质

1. 纸张　2. 塑料　3. 金属（以编号 ID 表示）

特征维度 3

价格

以浮点数表示，单位为千元。例如 59 元表示为 0.059。

- 《三体》：价格 59 元，类目为书本（ID 1），材质为纸张（ID 1），特征向量为 $(1, 1, 0.059)$。
- 《人工智能算法》：价格 69 元，类目为书本（ID 1），材质为纸张（ID 1），特征向量为 $(1, 1, 0.069)$。
- 电脑：价格 5000 元，类目为电子设备（ID 2），材质为金属（ID 3），特征向量为 $(2, 3, 5.000)$。

嵌入向量实例与相似度

表 1.4　嵌入向量实例

编号	物品名称	类目	材质	价格	特征向量
1	《三体》	书本:1	纸张:1	0.059	$(1, 1, 0.059)$
2	《人工智能算法》	书本:1	纸张:1	0.069	$(1, 1, 0.069)$
3	电脑	电子设备:2	金属:3	5.000	$(2, 3, 5.000)$

- 通过计算特征向量之间的相似度（如余弦相似度），物品 1（《三体》）与物品 2（《人工智能算法》）的相似度较高，而物品 1 与物品 3（电脑）的相似度较低。
- 物品实体基于人工设计的特征映射为特征向量；类似地，每个词元也可以用一个特征向量来表示，这种向量被称为词嵌入向量。

从显式特征到隐式表征

显式特征

对于物品实体，能够显式定义特征向量中特定维度的含义，如词性、是否为人名、物种类别、是否为地名等属性。

隐式表征

对于高维词嵌入向量，通常难以显式定义每个维度的具体含义，因此需要通过学习来获得词元的隐式表征。

随机初始化

先为每个词元分配随机向量

→

设定优化目标

如 Word2Vec 的对数似然

→

学习得到词嵌入

每个维度的含义由数据决定

词嵌入矩阵：从编号到向量

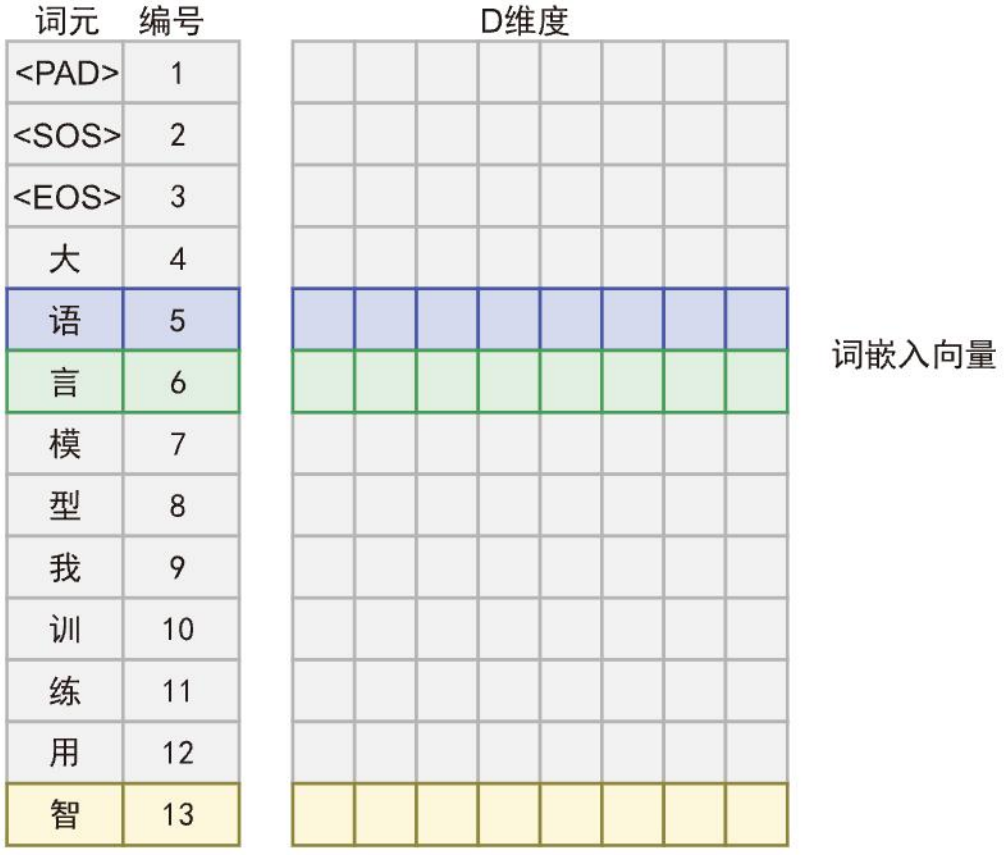


图 1.8　词元编号到词嵌入的映射

- 按照预设的维度创建词嵌入矩阵，每个词元编号对应矩阵中的一行向量。
- 给定一个词元编号序列，根据每个编号索引矩阵中对应的行，即可取得词嵌入向量。
- 由此实现从词元编号到词嵌入向量的转化。

图源 《大语言模型：训练、精调与对齐》图 1.8

Word2Vec：利用词的共现学习向量

Word2Vec（Word to Vector）利用词元在相邻文本中的共现关系来学习其语义表示。它基于一个关键假设：在文本中相邻出现的词元，其在向量空间中的表示也应彼此接近。

训练方式 1

连续词袋模型（CBOW）

根据给定窗口内的上下文词语来预测中心词。

训练方式 2

连续 Skip-gram 模型

根据中心词来预测其窗口内的上下文词语。

- 模型的目标是使一个词能够有效预测其周围上下文窗口内的其他词（或反之），在大规模语料上进行自监督学习。
- 训练得到的词向量（词嵌入）可广泛应用于下游自然语言处理任务。本书以 Skip-gram 模型为例描述算法原理。

文献 Mikolov et al., *Efficient Estimation of Word Representations in Vector Space*, 2013

Skip-gram：滑动窗口构造训练样本

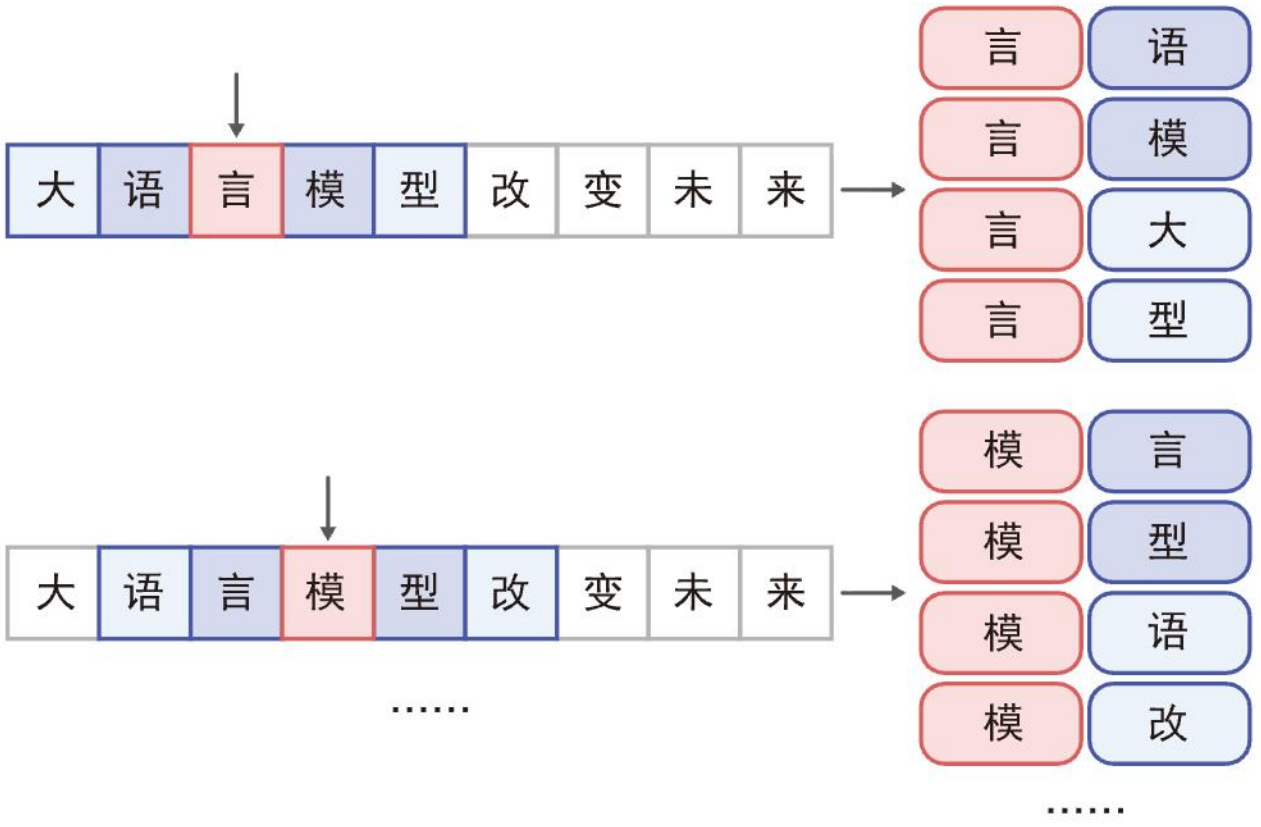


图 1.9　Word2Vec 的滑动窗口构造训练样本

- 语料 C（Corpus）中的每个单词 $w \in \mathcal{V}$（词表 Vocabulary）对应一个 d 维向量 $\boldsymbol{v}_w \in \mathbb{R}^d$，目标是学习这些向量。
- 窗口大小为 4 时，上下文位置集为 $N = \{\pm 1, \pm 2, \pm 3, \pm 4\}$。
- 中心词为“言”时，预测其上下文窗口内的词元为“语”“模”“大”和“型”。

图源 《大语言模型：训练、精调与对齐》图 1.9

条件概率：Softmax 形式

中心词 w_i 预测窗口内的每个上下文词 w_j，条件概率 $p(w_j|w_i)$ 由 Softmax 给出。

$$p(w_j|w_i) = \frac{\exp(\boldsymbol{v}_{w_i}\boldsymbol{v}_{w_j}^T)}{\sum_{w\in\mathcal{V}}\exp(\boldsymbol{v}_w\boldsymbol{v}_{w_i}^T)}$$

式 (1-7)：$\boldsymbol{v}_{w_i}\boldsymbol{v}_{w_j}^T$ 为两词向量的点积（标量，衡量相似度）；分母对词表 $\mathcal{V}$ 中所有单词求和，是归一化项。

$$s_w = \boldsymbol{v}_{w_i}\boldsymbol{v}_w^T = \boldsymbol{v}_w\boldsymbol{v}_{w_i}^T, \quad w \in \mathcal{V}$$

$$p(w_j|w_i) = \frac{\exp(s_{w_j})}{\sum_{w\in\mathcal{V}}\exp(s_w)}$$

步骤 1–2：中心词 w_i 固定，点积满足交换律，故 s_w 是标量；分子取 s_{w_j}，分母对全部 s_w 求和，即 Softmax 在 w_j 处的取值。

$$\sum_{w_j\in\mathcal{V}} p(w_j|w_i) = \frac{\sum_{w_j\in\mathcal{V}}\exp(s_{w_j})}{\sum_{w\in\mathcal{V}}\exp(s_w)} = 1$$

步骤 3：分子求和与分母相同，故 $p(\cdot|w_i) \in \mathbb{R}^{|\mathcal{V}|}$ 是词表上的概率分布。

对数似然与优化目标：式 (1-8)–(1-10)

为便于优化，通常将最大化联合概率 $\mathcal{L}$ 的目标转化为等价的最大化对数似然 $\mathcal{L}_{\text{MLE}}$；代入式 (1-7) 后即可化为可优化的形式。

$$\mathcal{L} = \prod_{i \in C} p(w_j | w_i), \quad j \in N + i$$

式 (1-8)：对语料 C 中所有（中心词 w_i，上下文词 w_j）的概率连乘。

$$\begin{aligned} \log \mathcal{L} &= \log \prod_{i \in C} \prod_{j \in N+i} p(w_j | w_i) \\ &= \sum_{i \in C,\, j \in N+i} \log p(w_j | w_i) \end{aligned}$$

步骤 1：对数严格递增，最大化 $\log \mathcal{L}$ 与最大化 $\mathcal{L}$ 等价；乘积的对数等于对数之和。

$$\mathcal{L}_{\text{MLE}} = \sum_{i \in C,\, j \in N+i} \log p(w_j | w_i)$$

式 (1-9)：对数似然，即 $\mathcal{L}$ 的对数形式。

$$\begin{aligned} \log p(w_j | w_i) &= \log \exp(\boldsymbol{v}_{w_i} \boldsymbol{v}_{w_j}^T) - \log \sum_{w \in \mathcal{V}} \exp(\boldsymbol{v}_w \boldsymbol{v}_{w_i}^T) \\ &= \boldsymbol{v}_{w_i} \boldsymbol{v}_{w_j}^T - \log \sum_{w \in \mathcal{V}} \exp(\boldsymbol{v}_w \boldsymbol{v}_{w_i}^T) \end{aligned}$$

步骤 2：将式 (1-7) 代入，商的对数等于对数之差，再由 $\log \exp(x) = x$ 化简。

$$\boldsymbol{v}_w = \arg \max_{\boldsymbol{v}_w,\, w \in \mathcal{V}} \mathcal{L}_{\text{MLE}}$$

式 (1-10)：求一组词向量 $\{\boldsymbol{v}_w \in \mathbb{R}^d \mid w \in \mathcal{V}\}$，使 $\mathcal{L}_{\text{MLE}}$ 最大化。

- 每个对数项 $\log p(w_j | w_i)$ 的分母都要遍历词表 $\mathcal{V}$ 中的所有单词。
- 当词表规模 $|\mathcal{V}|$ 很大时计算开销巨大，因此需要引入负采样来提高训练效率。

负采样：减少每次更新的点积次数

负采样中，Word2Vec 不再直接计算完整词表上所有词的内积的指数和（即 Softmax 的分母项）。



图 1.10　Word2Vec 增加负采样提升训练效率

- 对正样本对（中心词 w_i，上下文词 w_j），从词表 $\mathcal{V}$ 随机采样 K 个负样本 w_k（通常不在 w_i 的上下文中）。
- 目标转变为区分中心词与正、负样本的搭配（通常使用二元逻辑回归损失）。
- 点积次数从 $O(|\mathcal{V}|)$ 降到 $O(K+1)$；$|\mathcal{V}| = 128\,000$、$K = 3$ 时，每次更新只需 4 次点积（1 次正样本 + 3 次负样本）。

图源 《大语言模型：训练、精调与对齐》图 1.10

负采样的训练目标

正样本取 $D=1$、负样本取 $D=0$，于是训练目标就是让这一组搭配的似然最大；取对数后连乘变为求和。

$$\log\left[\sigma(\boldsymbol{v}_{w_i}\boldsymbol{v}_{w_j}^T)\prod_{k=1}^{K}\sigma(-\boldsymbol{v}_{w_i}\boldsymbol{v}_{w_k}^T)\right]=\log\sigma(\boldsymbol{v}_{w_i}\boldsymbol{v}_{w_j}^T)+\sum_{k=1}^{K}\log\sigma(-\boldsymbol{v}_{w_i}\boldsymbol{v}_{w_k}^T)$$

一个正样本 (w_i,w_j) 与 K 个负样本 (w_i,w_k) 的似然连乘取对数；训练时最大化该目标，等价于最小化其负值作为损失。

正样本

取 $D=1$，概率为 $\sigma(\boldsymbol{v}_{w_i}\boldsymbol{v}_{w_j}^T)$，其中 $\sigma(x)=\frac{1}{1+e^{-x}}$ 为 logistic 函数。

负样本

取 $D=0$，由 $1-\sigma(x)=\sigma(-x)$，概率为 $\sigma(-\boldsymbol{v}_{w_i}\boldsymbol{v}_{w_k}^T)$。

计算量

与式 (1-7) 对整个词表求和相比，负采样每次只需 $K+1$ 次点积。

文献 Goldberg & Levy, *word2vec Explained: deriving Mikolov et al.'s negative-sampling word-embedding method*, 2014

预训练词向量的使用



预训练的词向量可以直接用于将词元序列转化为对应的词向量序列，这些序列表示可以作为输入特征，应用于各种下游的自然语言处理任务。

词嵌入层

现代主流的深度学习语言模型通常包含一个嵌入层（Embedding Layer），将离散的词元（通常表示为索引）映射为连续的稠密向量，即词嵌入。该层的参数是一个词嵌入矩阵，每一行对应词表中一个词元的词嵌入，初始化时可以随机初始化。



训练方式

在目标任务中端到端优化

嵌入矩阵不必预先独立训练，可直接在文本分类或机器翻译等任务的端到端训练中优化。损失的梯度经反向传播逐层传递到嵌入层，更新词嵌入矩阵的参数。

参数初始化

随机初始化或预训练初始化

嵌入层参数可用预训练的词向量初始化，此时预训练向量提供良好的起点；这些参数既可以在后续任务训练中保持冻结，也可以继续微调，以更好地适应目标任务。

从词到句：静态词向量的局限

Word2Vec 与词嵌入层得到的是**静态**词向量：词表中每个词只有一行参数，同一个词无论出现在哪个句子里，查表取到的向量完全相同。但词义由上下文决定。

句子	“苹果”的语义	静态向量
我吃了一个苹果。	一种水果	同一行 $\boldsymbol{v}_{\text{苹果}}$
苹果发布了新款手机。	一家公司	
他买了一台苹果电脑。	一个品牌（修饰“电脑”）	

静态词向量只能学到三种用法的“折中”表示，无法区分它们在句中的差别。那么，如何由词到句、描述整句的语义？

词向量求平均

$\frac{1}{n}\sum_i \boldsymbol{v}_{w_i}$ 简单，但丢失词序：“狗咬人”与“人咬狗”平均后相同。

加权组合

让每个词的权重随句子变化，但权重从何而来？固定权重仍与位置无关。

按上下文动态表征

同一个词在不同句子中生成不同向量，权重由它与句中其他词的关系算出。

下一步 第三种做法的权重就是注意力分数：每个词根据与句中其他词的相关程度汇聚信息，从而得到上下文相关的表征。这正是第 2 章 Transformer 的出发点。

1.5 小结

回顾大语言模型的发展历程与文本数据处理方法，并指向后续章节的展开。

本节内容

本章回顾

后续章节

本章回顾

本章梳理了大语言模型的发展历程，并引入了基础的文本数据处理方法。

缩放定律

规模增长，性能提升

缩放定律实验表明，随着参数规模、计算量和数据量的增长，预训练模型的性能会相应提升。

涌现现象

规模超过阈值，能力跃升

当模型规模超过一定阈值时，会产生“涌现”现象，模型的通用能力能够得到巨大的提升。

预训练范式

纯解码器 Transformer

采用纯解码器 Transformer 架构的语言模型经过通用语料预训练后，能够学习到强大的通用语言表征，这一范式对语言模型的发展起到了关键作用。

文本数据处理

分词与嵌入

分词和嵌入是基础的文本数据处理方法，为后续展开讲解模型架构和算法原理奠定基础。

后续章节的展开

本书将依次讲解 Transformer、GPT 和 Llama 模型，涉及模型结构、语言建模、实现细节和工程技巧；随后介绍高效微调与应用，再到强化学习与分布式训练部署。

第 2–4 章 · 模型

Transformer、GPT 与 Llama

注意力机制与模型结构、语言建模任务、关键模块的实现细节，以及工程技巧。

第 5–6 章 · 应用

高效微调与模型应用

参数高效微调方法（如 LoRA），以及提示词、检索增强等应用方式。

第 7–8 章 · 对齐

强化学习与 RLHF

强化学习的基本理论，以及偏好对齐、可验证奖励下的推理强化学习。

第 9–10 章 · 工程

分布式训练与部署优化

显存与并行策略、混合并行训练系统，以及推理加速与高吞吐服务。

四组内容构成"模型 → 应用 → 对齐 → 工程"的递进：先把模型读懂，再学会用得省，最后把训练与推理跑在真实集群上。

思考题与实践题

1 人是如何理解句子的？与语言模型“基于历史词元预测下一个词元”有何异同？

2 如何用特征表达句子的语义？词向量求平均、加权组合、按上下文动态表征各有什么代价？

3 无监督（自监督）训练相比有监督训练有什么优势？为什么大语言模型能以文本自身作为标签？

4 影响模型表现的重要因素有哪些？缩放定律中的 C、D、N 如何共同决定损失？

5 语言模型如何使用工具？提示词、函数调用与外部检索各自解决什么问题？

6 如何让语言模型理解图片？把视觉信息接入语言模型需要解决哪一步对齐？

7 实践：实现一个 Word2Vec skip-gram 的前向与反向算法，用负采样损失训练，并画出训练后词向量的二维分布。

8 实践：实现一个字符分词器（词表构建、编码、解码），再增加规则（如合并高频二元组）以减少分词后的词元长度，并比较前后长度。